

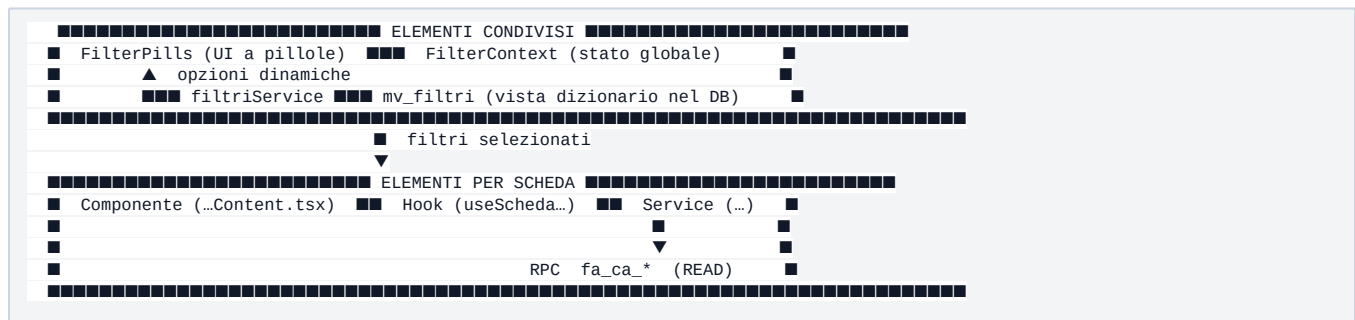
Nota di Metodo — Filtri e schede del Conto Annuale

Documento illustrativo del **metodo** con cui realizziamo le schede della dashboard: filtri dinamici centralizzati + tre livelli riutilizzabili (**Service** → **Hook** → **Componente**). Gli esempi sono tratti dal codice reale della scheda "Analisi per età".

Principio operativo: l'applicazione **non scrive nulla** sul database del cliente. Legge i menu dei filtri dalla vista `mv_filtri` e invoca **funzioni RPC pre-calcolate** (`fa_ca_*`) in **sola lettura**.

1. Il metodo in sintesi

Ogni scheda nasce dalla composizione di **quattro elementi**, tre dei quali sono **condivisi da tutte le schede** (i filtri) e tre specifici (service, hook, componente):



Per una **nuova scheda** ricalchiamo **Service** → **Hook** → **Componente**; i filtri si **ereditano gratis**. Questo è il motivo per cui procediamo rapidamente.

2. I filtri (elemento condiviso)

2.1 Lo stato globale — `contexts/FilterContext.tsx`

Un contenitore React che conserva i filtri correnti e li mette a disposizione di tutta la dashboard. I valori "neutri" sono stringhe ("**Tutti**" / "**Tutte**"), l'anno di default è "**2023**".

```
export interface FilterState {
  macrocategoria: string; categoria: string; comparto: string;
  regione: string; genere: string; anno: string;
  dimensione_pa: string; cluster: string;
}
const defaultFilters = { macrocategoria:"Tutte", categoria:"Tutte", comparto:"Tutti",
  regione:"Tutte", genere:"Tutti", anno:"2023", dimensione_pa:"Tutte", cluster:"Tutti" };

export const FilterProvider = ({ children }) => {
  const [filters, setFilters] = useState(defaultFilters);
  const setFilter = (key, value) => setFilters(prev => ({ ...prev, [key]: value }));
  const resetFilters = () => setFilters(defaultFilters);
  // quante pillole sono diverse dal default (per il badge "Reset (n)")
  const activeCount = Object.entries(filters)
    .filter(([k, v]) => v !== defaultFilters[k]).length;
  return <FilterContext.Provider value={{ filters, setFilter, resetFilters, activeCount }}>
    {children}</FilterContext.Provider>;
};
```

Chi consuma i filtri usa un semplice hook:

```
const { filters, setFilter, resetFilters, activeCount } = useFilters();
```

2.2 Le opzioni dei filtri dal DB — `services/ca/filtriService.ts`

I menu **non sono cablati nel codice**: vengono letti dalla vista dizionario `mv_filtri` (partizionata per anno, con relazione padre/figlio per la cascata).

```
async function query(tipo, anno, chiavePadre?) {
  let q = sbUntyped.from("mv_filtri")
    .select("chiave, codice, descrizione, chiave_padre")
    .eq("tipo", tipo).eq("anno", anno)
    .order("ordine").order("descrizione");
```

```

if (chiavePadre) q = q.eq("chiave_padre", chiavePadre);
const { data, error } = await q;
if (error) throw error;
return (data ?? []).map(/* ... pulizia descrizioni ... */);
}
export const fetchComparti = (anno) => query("comparto", anno);
export const fetchMacrocategorie = (anno, chiaveComparto) => query("macrocategoria", anno, chiaveComparto);
export const fetchCategorie = (anno, chiaveMacro) => query("categoria", anno, chiaveMacro);
export const fetchRegioni = (anno) => query("regione", anno);

```

2.3 La UI a pillole — `components/dashboard/FilterPills.tsx`

a) Opzioni caricate dinamicamente (React Query), con `enabled` per la cascata:

```

const compartiQ = useQuery({ queryKey: ["mvf", "comparti", anno], queryFn: () => fetchComparti(anno) });
const macroQ = useQuery({
  queryKey: ["mvf", "macro", anno, filters.comparto],
  queryFn: () => fetchMacrocategorie(anno, `comparto:${filters.comparto}`),
  enabled: filters.comparto !== "Tutti", // le macro si caricano solo con un comparto scelto
});
// la categoria dipende dalla CHIAVE della macro selezionata (non dal codice)
const macroChiave = (macroQ.data ?? []).find(m => m.codice === filters.macrocategoria)?.chiave;
const categorieQ = useQuery({
  queryKey: ["mvf", "cat", anno, macroChiave],
  queryFn: () => fetchCategorie(anno, macroChiave),
  enabled: Boolean(macroChiave),
});

```

b) **Cascata con reset a catena.** Cambiando Comparto si azzerano Macro e Categoria; cambiando Anno si azzerava tutta la cascata (perché `mv_filtr` è partizionata per anno):

```

// Comparto
onChange={(v) => { setFilter("comparto", v);
  setFilter("macrocategoria", "Tutte");
  setFilter("categoria", "Tutte"); }}
// Anno
onChange={(v) => { setFilter("anno", v); setFilter("comparto", "Tutti");
  setFilter("macrocategoria", "Tutte"); setFilter("categoria", "Tutte"); }}

```

c) **Adattamento al ruolo.** Gli utenti `ente_hr` non vedono Comparto/Regione (il loro perimetro dati è vincolato all'ente):

```

const isEnteHr = profile?.role === "ente_hr";
{!isEnteHr && (< /* pillole Comparto e Regione */ </>)}

```

d) **Reset ed esporta.** Il badge "Reset (n)" appare solo se `activeCount > 0`; a destra un pulsante "Esporta" e la data di riferimento.

Ogni pillola è lo stesso componente `Pill` riusato: mostra `label` quando è neutra, `label: valore` quando è attiva, con la "x" per azzerarla.

3. Il Service (per scheda) — `services/ca/analisiEtaService.ts`

Unico strato che parla col DB: dichiara i campi attesi e invoca le RPC.

Passo 1 — contratto dei filtri (tipizzato):

```

export interface EtaFiltri {
  anno: number; istituzione?: string|null; codiceFiscale?: string|null;
  comparto?: string|null; macrocategoria?: string|null; categoria?: string|null;
  regione?: string|null; genere?: "T"|"U"|"D"|null;
}

```

Passo 2 — la regola d'oro dei parametri (`buildParams`): un parametro entra solo se ha valore (una "" azzererebbe i risultati lato DB):

```

function buildParams(f, opts = {}) {
  const p = {};
  if (f.anno !== null) p.p_anno = f.anno;
  if (f.istituzione) p.p_istituzione = f.istituzione; // omissso se non è un ente
  if (f.comparto) p.p_comparto = f.comparto;
  if (f.macrocategoria) p.p_macrocategoria = f.macrocategoria;
  if (f.categoria) p.p_categoria = f.categoria;
  if (f.regione) p.p_regione = f.regione;
  if (opts.includeGenere !== false && f.genere && f.genere !== "T") p.p_genere = f.genere;
  return p;
}

```

Passo 3 — helper di invocazione (riga singola per i KPI, lista per grafici):

```
async function rpcRows(fn, params) { const {data,error}=await sbUntyped.rpc(fn,params);
  if (error) throw error; return data ?? []; }
async function rpcOne(fn, params) { return (await rpcRows(fn, params))[0] ?? null; }
```

Passo 4 — una funzione per ogni elemento a schermo (i casi particolari restano isolati e dichiarati):

```
export const fetchPersonaleServizio = (f) => rpcOne("fa_ca_personale_servizio", buildParams(f));
export const fetchEtaCard = (f) => rpcOne("fa_ca_eta", buildParams(f));
export const fetchFasceGenere = (f) => rpcRows("fa_ca_eta_fasce_genere", buildParams(f, { includeGenere:false }));
export const fetchEvoluzione = (f, genereToggle) =>
  rpcRows("fa_ca_eta_evoluzione", buildParams({ ...f, genere: genereToggle }));
export const fetchBenchmark = (f, dimensione) =>
  rpcRows("fa_ca_eta_benchmark", { ...buildParams(f), p_dimensione: dimensione });
```

4. L'Hook (per scheda) — [hooks/useAnalisiEta.ts](#)

Hook sottili che incapsulano cache e stato asincrono. La `queryKey` include i filtri: al loro cambio, React Query rifà la chiamata e la UI si aggiorna da sola.

```
export const usePersonaleServizio = (f) =>
  useQuery({ queryKey: ["ca-eta","personale", f], queryFn: () => fetchPersonaleServizio(f) });
export const useFasceGenere = (f) =>
  useQuery({ queryKey: ["ca-eta","fasce", f], queryFn: () => fetchFasceGenere(f) });
// ... un hook per ciascuna funzione del service ...
```

5. Il Componente (per scheda) — [components/dashboard/AnalisiEtaContent.tsx](#)

Strato di presentazione: legge i filtri, li traduce nel contratto del service, chiede i dati agli hook e disegna KPI/grafici/tabelle.

Passo 1 — legge stato e ruolo:

```
const { filters } = useFilters();
const { profile } = useAuth();
const isDfp = profile?.role === "dfp"; // il selettore Ente è solo del DFP
const [serieGenere, setSerieGenere] = useState("T"); // toggle locale del grafico
```

Passo 2 — il "ponte" tra UI e service (valori neutri → null):

```
const gMap = { Tutti:"T", Uomini:"U", Donne:"D" };
const filtri = {
  anno: Number(filters.anno) || 2023,
  istituzione: ente?.codice ?? null,
  comparto: filters.comparto !== "Tutti" ? filters.comparto : null,
  macrocategoria: filters.macrocategoria !== "Tutte" ? filters.macrocategoria : null,
  categoria: filters.categoria !== "Tutte" ? filters.categoria : null,
  regione: filters.regione !== "Tutte" ? filters.regione : null,
  genere: gMap[filters.genere] ?? "T",
};
```

Passo 3 — chiede i dati (un hook per elemento):

```
const personale = usePersonaleServizio(filtri); // KPI
const fasce = useFasceGenere(filtri); // piramide + tabella
const evo = useEvoluzione(filtri, serieGenere); // grafico con toggle locale
```

Passo 4 — rende robusti i dati e disegna:

```
const fasceRows = fasce.data ?? []; // mai undefined
// KPI cards (componente Kpi riusato), grafici Recharts, tabella con Δ colorato,
// selettore Ente (solo DFP) con autocomplete che pilota p_istituzione.
```

Elementi condivisi dentro il componente: helper di formattazione italiana (`nf`, `n1`), gestione dei valori nulli (`-`), card KPI e selettori riutilizzabili.

6. Flusso end-to-end (dal clic al dato)

1. L'utente apre la pillola "Comparto" → FilterPills carica le opzioni da mv_filtri
2. Sceglie un valore → setFilter aggiorna FilterContext e azzera la cascata
3. Il Componente ricalcola l'oggetto "filtri" (neutri → null)
4. Cambia la queryKey degli Hook → React Query rifà le chiamate
5. Il Service costruisce i parametri (buildParams) e chiama la RPC fa_ca_* (READ)
6. Arrivano i dati già calcolati → KPI, grafici e tabelle si aggiornano

7. Perché "andiamo spediti" (e i limiti onesti)

Acceleratori - Separazione netta dati/UI: si cambia l'uno senza toccare l'altro. - **Riuso massiccio**: filtri, selettore Ente, formattazioni, buildParams e card sono condivisi → una nuova scheda è **composizione**, non riscrittura. - **Backend che calcola**: nessuna aggregazione lato front-end → meno bug. - **Aggiornamento automatico**: React Query ricarica al cambio filtro.

Condizioni (lato cliente) - Un **Excel di mappatura per scheda** con RPC, parametri, campi di output e **le note/postille** (i casi particolari). - Le **RPC già disponibili e funzionanti** sul database.

Limiti da non nascondere - Le **note vanno lette e recepite caso per caso** (es. parametri fissi, genere applicato in modo non uniforme, saldi negativi, formati "lungo" con pivot). - Se una RPC manca o cambia i nomi dei campi, i tempi dipendono dal backend. - Grafici o requisiti UX fuori standard possono richiedere lavoro extra.

8. Risultato ad oggi

Con questo metodo sono già state portate su dati reali **6 schede**: Analisi per età, Anzianità di servizio, Assunti per causale, Cessazioni dal servizio, Tasso di turnover, Tasso di sostituzione — tutte con **filtri dinamici condivisi** e in **sola lettura** sul database del cliente.